

HPC- DFS

```
#include <iostream>
#include <vector>
#include <queue>
#include <omp.h>
using namespace std;

class Graph {
public:
    int V;
    vector<vector<int>>> adj;
    vector<bool> visited;

    Graph(int V) {
        this->V = V;
        adj.resize(V);
        visited.resize(V, false);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void parallelDFS(int node) {
        #pragma omp critical
        {
            if (visited[node]) return;
            visited[node] = true;
            cout << node << " ";
        }

        #pragma omp parallel for
        for (int i = 0; i < adj[node].size(); i++) {
            int neighbor = adj[node][i];
            if (!visited[neighbor]) {
                parallelDFS(neighbor);
            }
        }
    }
}
```

```

};

int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;
    cout << "Enter " << E << " edges (u v):\n";
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
    }

    int startNode;
    cout << "Enter starting node for DFS: ";
    cin >> startNode;
    cout << "Parallel DFS: ";
    g.parallelDFS(startNode);
    cout << endl;

    return 0;
}

```

o/p:

Enter number of vertices: 5

Enter number of edges: 4

Enter 4 edges (u v):

0 1

0 2

1 3

2 4

Enter starting node for DFS: 0

Parallel DFS: 0 1 3 2 4

HPC- BFS

```
#include <iostream>
#include <vector>
#include <queue>
#include <omp.h>
using namespace std;

class GraphBFS {
public:
    int V;
    vector<vector<int>> adj;

    GraphBFS(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void parallelBFS(int start) {
        vector<bool> visited(V, false);
        queue<int> q;

        visited[start] = true;
        q.push(start);

        cout << "Parallel BFS: ";

        while (!q.empty()) {
            int qSize = q.size();

            #pragma omp parallel for
            for (int i = 0; i < qSize; i++) {
                int node;
                #pragma omp critical
                {
                    if (!q.empty()) {
                        node = q.front();
                        q.pop();
                        cout << node << " ";
                    }
                }
            }
        }
    }
};
```

```

    }

    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            #pragma omp critical
            {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }
    }
}

cout << endl;
}
};

int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    GraphBFS g(V);

    cout << "Enter number of edges: ";
    cin >> E;
    cout << "Enter " << E << " edges (u v):\n";
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
    }

    int startNode;
    cout << "Enter starting node for BFS: ";
    cin >> startNode;

    g.parallelBFS(startNode);

    return 0;
}

```

o/p:

Enter number of vertices: 7

Enter number of edges: 6

Enter 6 edges (u v):

0 1

0 2

1 3

1 4

2 5

2 6

Enter starting node for BFS: 0

Parallel BFS: 0 1 2 3 4 5 6

HPC- Merge sort

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <chrono>

using namespace std;
using namespace std::chrono;

void takeUserInputArray(vector<int>& arr, int size) {
    cout << "Enter " << size << " elements:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }
}

void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int i = 0; i < n2; i++) R[i] = arr[mid + 1 + i];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void parallelMergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        #pragma omp parallel sections
        {
            #pragma omp section
            parallelMergeSort(arr, left, mid);
            #pragma omp section
            parallelMergeSort(arr, mid + 1, right);
        }

        merge(arr, left, mid, right);
    }
}
```

```

}

int main() {
    int n;
    cout << "Enter the size of the array: ";
    cin >> n;

    vector<int> arr(n);
    takeUserInputArray(arr, n);

    auto start = high_resolution_clock::now();
    parallelMergeSort(arr, 0, n - 1);
    auto stop = high_resolution_clock::now();
    auto parMergeTime = duration_cast<milliseconds>(stop - start).count();

    // Displaying the sorted array
    cout << "Sorted Array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Displaying the execution time for parallel merge sort
    cout << "Parallel Merge Sort Execution Time: " << parMergeTime << " ms\n";

    return 0;
}

```

o/p:

Enter the size of the array: 5

Enter 5 elements:

10

5

2

9

7

Sorted Array: 2 5 7 9 10

Parallel Merge Sort Execution Time: 0 ms

HPC- Bubble sort

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <chrono>

using namespace std;
using namespace std::chrono;

void takeUserInputArray(vector<int>& arr, int size) {
    arr.resize(size);
    cout << "Enter " << size << " elements:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }
}

void printArray(const vector<int>& arr) {
    for (int val : arr) {
        cout << val << " ";
    }
    cout << endl;
}

void parallelBubbleSort(vector<int>& arr, int n) {
    for (int i = 0; i < n; i++) {
        int start = i % 2;
        #pragma omp parallel for
        for (int j = start; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

int main() {
    int n;
    cout << "Enter the size of the array: ";
    cin >> n;

    vector<int> arr(n);

    takeUserInputArray(arr, n);
```



```

// Measure execution time for parallel bubble sort
auto start = high_resolution_clock::now();
parallelBubbleSort(arr, n);
auto stop = high_resolution_clock::now();
auto parBubbleTime = duration_cast<milliseconds>(stop - start).count();

// Display the sorted array
cout << "\nSorted array using Parallel Bubble Sort:\n";
printArray(arr);

// Display the execution time for parallel bubble sort
cout << "\nParallel Bubble Sort Execution Time: " << parBubbleTime << " ms\n";

return 0;
}

```

o/p:

Enter the size of the array: 5

Enter 5 elements:

9

6

8

5

2

Sorted array using Parallel Bubble Sort:

2 5 6 8 9

Parallel Bubble Sort Execution Time: 0 ms

HPC-Min, Max, Sum, Average

```
#include <omp.h>
#include <iostream>
#include <chrono>
using namespace std::chrono;
using namespace std;

// Display Array
void displayArray(int nums[], int length)
{
    cout << "Nums: [";
    for (int i = 0; i < length; i++) {
        cout << nums[i];
        if (i != length - 1)
            cout << ", ";
    }
    cout << "]" << endl;
}

// Parallel Min Operation
void minOperation(int nums[], int length)
{
    int minValue = nums[0];
    #pragma omp parallel for reduction(min : minValue)
    for (int i = 0; i < length; i++) {
        minValue = (nums[i] < minValue) ? nums[i] : minValue;
    }
    cout << "Min value: " << minValue << endl;
}

// Parallel Max Operation
void maxOperation(int nums[], int length)
{
    int maxValue = nums[0];
    #pragma omp parallel for reduction(max : maxValue)
    for (int i = 0; i < length; i++) {
        maxValue = (nums[i] > maxValue) ? nums[i] : maxValue;
    }
    cout << "Max value: " << maxValue << endl;
}

// Parallel Sum Operation
void sumOperation(int nums[], int length)
{

```

```

    int sum = 0;
#pragma omp parallel for reduction(+ : sum)
    for (int i = 0; i < length; i++) {
        sum += nums[i];
    }
    cout << "Sum: " << sum << endl;
}

```

```

// Parallel Average Operation
void avgOperation(int nums[], int length)
{
    float sum = 0;
#pragma omp parallel for reduction(+ : sum)
    for (int i = 0; i < length; i++) {
        sum += nums[i];
    }
    cout << "Average: " << (sum / length) << endl;
}

```

```

// Main Function
int main()
{
    int length;
    cout << "Enter number of elements: ";
    cin >> length;

    int* nums = new int[length]; // dynamic allocation

    cout << "Enter elements:\n";
    for (int i = 0; i < length; i++) {
        cin >> nums[i];
    }

    auto start = high_resolution_clock::now();

    displayArray(nums, length);
    minOperation(nums, length);
    maxOperation(nums, length);
    sumOperation(nums, length);
    avgOperation(nums, length);

    auto stop = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(stop - start);

    cout << "\nExecution time: " << duration.count() << " microseconds" << endl;
}

```

```
delete[] nums; // free memory

return 0;
}
```

o/p:

Enter number of elements: 5

Enter elements:

1

10

9

5

20

Nums: [1, 10, 9, 5, 20]

Min value: 1

Max value: 20

Sum: 45

Average: 9

Execution time: 88 microseconds